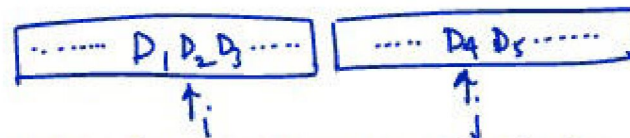


- After the recursive MS calls,
if right most element of left half
 $\leq$ left most element of right half
then array is sorted $\Rightarrow$ no need to merge
- Best case when array is already sorted,
MS takes $\Theta(n)$ times
- Create one aux array at the start of MS & use
that as extra space for the merge step
- We could have a cutoff to insertion sort
(7 elements is the sweet spot).
- For a comparison based sorting, in the worst case
running time is $\Omega(n \log n)$.

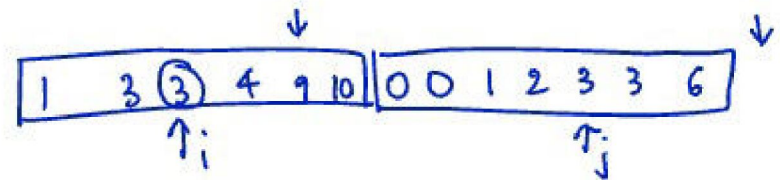
Inversions using merge sort

- 3 kinds of inversions:
 - contained within the left subarray } found recursively
 - contained within the right subarray }
 - split inversion
- Split inversions:
 - during merge step $\rightarrow$ sorting each of the subarrays won't change #split inversions
 - whenever there are duplicates we copy the element from the left



Split inversions corresponding to D_2
 $=$ #elements in right subarray
 that are smaller than D_2
 $=$ #times j was incremented

1, 3



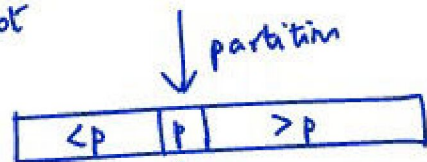
0 0 1 1 2 3

Radix sort (LSD to MSD)

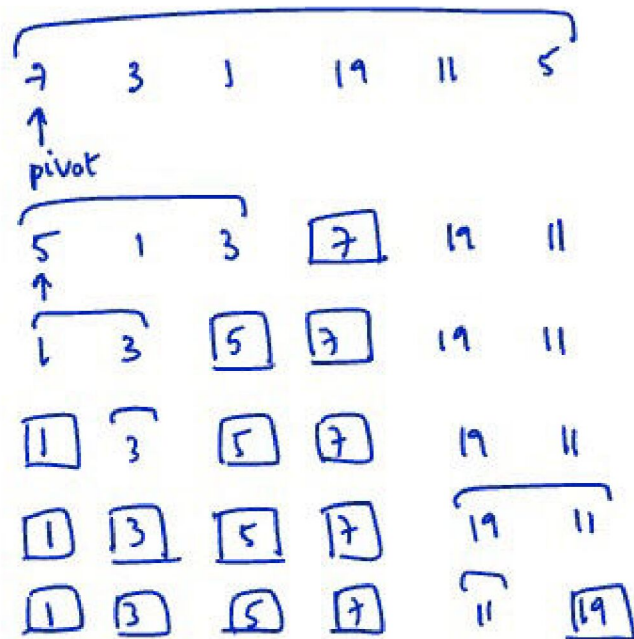
Quick sort

- Randomized algorithms

7 3 1 19 11 5
↑
pivot partition



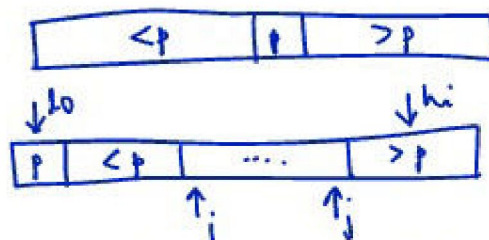
↓
this pivot goes to the place
where it's supposed to go
after the sort



Partition:

- Inplace

7 3 1 19 11 5


$$lo + 1 \text{ to } i - 1 : < p$$
$$j+1 \text{ to } \underline{hi} : > p$$

i starts at $low + 1$

j starts at hi

run as long as $i \leq j$

if name[i] < p:

```

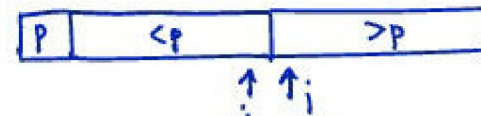
i++

```

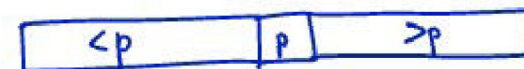
if $nums[i] > p$:

swap(i, j)

j:-



Swap $\text{nums}[lo] \{ \text{nums}[j] \}$



$$T(n) = 2T(n/2) + \underbrace{\Theta(n)}_{\substack{\text{partition} \\ \text{has to happen in} \\ \text{linear time}}}$$

- Given that the elements of the array are randomly ordered (uniformly random)

Expected RT = $O(n \log n)$

- If the pivot divides the array into a fraction of the original array we still get $O(n \log n)$ RT expected

$$\boxed{100} \mid \boxed{n-100} \quad \times$$

$$\boxed{0.1n} \mid \boxed{0.9n} \quad \text{Expected RT: } n \log n$$

- Worst case:

$$\begin{array}{cccccc} 19 & 11 & 7 & 5 & 3 & 1 \\ \uparrow & & & & & \\ 3 & 7 & 5 & 1 & 11 & \boxed{19} \end{array}$$

$$\# \text{ steps: } n + n-1 + n-2 + \dots = \Theta(n^2)$$

- Randomly shuffle the array once initially

↓
uniformly random
⇒ every permutation has same chance

Fisher Yates shuffle
linear time

Biased coin $P(H) = 0.25$
 $P(T) = 0.75$

- Game fee: \$1
- Heads: \$2
- Tails: don't get anything

$$\begin{aligned} \text{Expected winnings: } & 0.25(\$1) + 0.75(-\$1) \\ & = -\$0.5 \end{aligned}$$

Duplicates

- No duplicates:



- Duplicates:



Not a good idea
as it could create
imbalanced partition

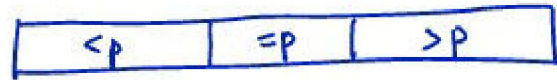
B A A B A B C C B C B
A B A B A B B B C C C

Idea 1:



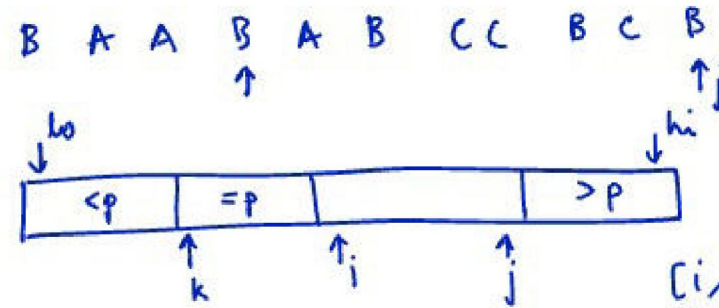
approximately half of duplicates go on each side

Idea 2:



- Dutch National flag partition
- better than $O(n \log n)$ when there are a lot of duplicates

- 3 Way partition



$[i, j]$ - range of elements we haven't seen yet

if $nums[i] < p$:

swap(i, k)

$k++$

$i++$

elif $nums[i] > p$:

swap(i, j)

$j--$

else:

$i++$

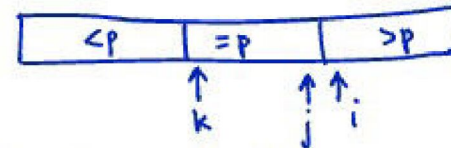
run as long as $i \leq j$

Initial values:

$i: lo + 1$

$j: hi$

$k: lo$



def $qs(nums, lo, hi)$:

$[p-lo, p-hi] = 3way\ partition(nums, lo, hi)$

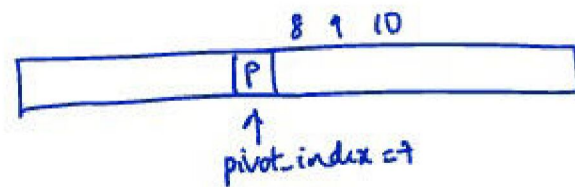
$qs(nums, lo, p-lo-1)$

$qs(nums, p-hi+1, hi)$

range of elements
= pivot

• Quick select: $k=10$

- Select the k^{th} largest element in the array



If $k > \text{pivot_index}$:

$qs(\text{nums}, \text{pivot_index}+1, \text{hi}, k)$

if $k < \text{pivot_index}$:

$qs(\text{nums}, \text{lo}, \text{pivot_index}-1, k)$

else:

return pivot

Quickselect

$$T(n) = T(n/2) + \underbrace{\Theta(n)}_{\text{partition}}$$

Average RT: $O(n)$

Worst case: $\Theta(n^2)$

• Getting ' k ' smallest elements in sorted order

① • k^{th} smallest using QS $\Rightarrow O(n)$

• run Qsort/msort on left half $\Rightarrow O(k \log k)$
 $O(n + k \log k)$

② Sort the whole array & get smallest $\Rightarrow O(n \log n)$

③ Min heap:

• Build the heap: $O(n)$

• Remove min ' k ' times: $O(k \log n)$

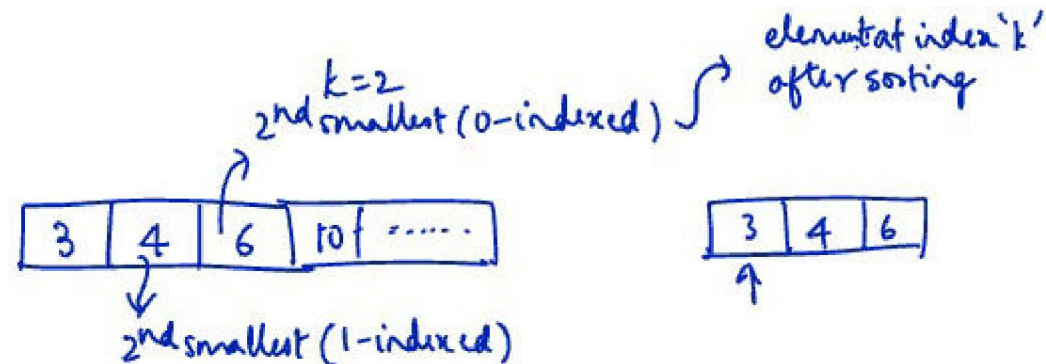
RT: $O(n + k \log n)$

• Getting ' k ' smallest in any order

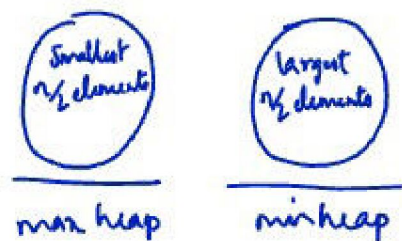
①, ②, ③ from above

④ • Run Quickselect(k)

all elements $< k$ will be on the left



- Find median in a stream
- Maintain two heaps



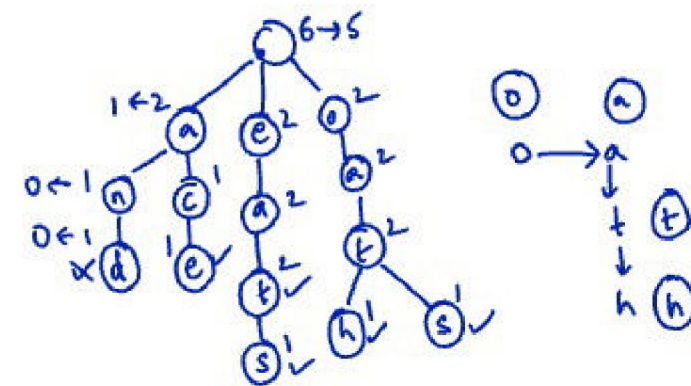
$$3 \quad 5 \quad 6 \quad 10 \quad \frac{5+6}{2} = 5.5$$

If n is even,

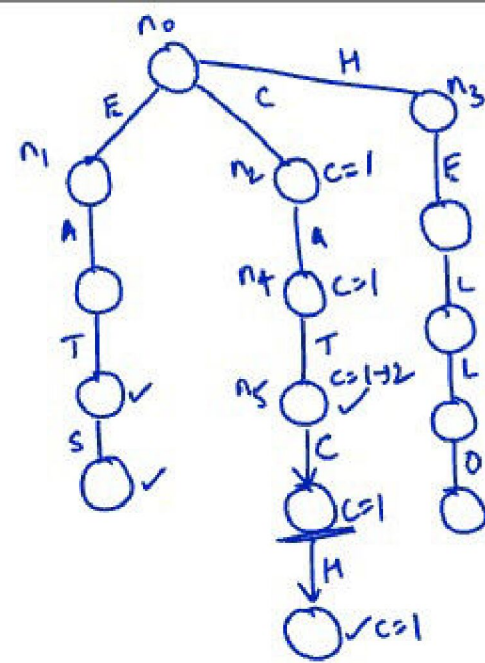
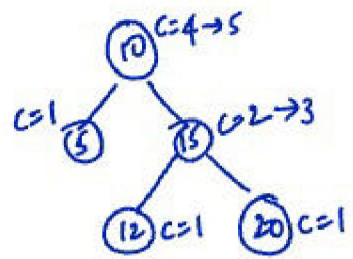
$$\text{median} = \frac{(\text{largest of the max heap} + \text{smallest of the min heap})}{2}$$

If n is odd,
 median is the extra element in max heap

Invariant: $\# \text{elements in max heap} - \# \text{elements in min heap} \leq 1$



lst = ["a", "b", "c"]
 "".join(lst)



```

insert(n0, CATCH, 0)
  OC=1
  insert(n2, CATCH, 1)
    OC=1
    insert(n4, CATCH, 2)
      OC=1
      insert(n5, CATCH, 3)
        OC=0
        child[C] = insert(None, CATCH, 4)
          OC=0
          insert(None, CATCH, 5)
            child[H] ←
  
```

words = [hot, dot, dog, lot, log, cog] W words
 0 1 2 3 4 5 L-word length

hot dot hat hda
 bot hbt hdb
 cot hct hcc
 ⋮ ⋮ ⋮
 zot hzt hcz

26 · L
 L × O(1) ↓
 getting neighbours

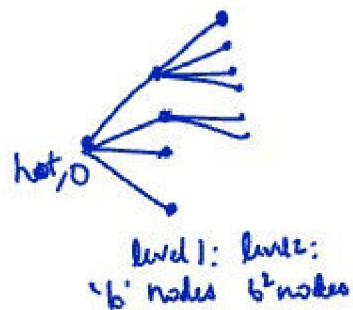
{ *ot : [hot, dot, lot]
 h*t : [hot,]
 ho* : [hot,]
 d*t : [dot,]
 do* : [dot, dog]
 *og : [dog, log, cog]
 d*g : [dog]
 l*t : [lot]
 lo* : [lot, log]
 l*g : [log]
 c*g : [cog]
 co* : [cog]

Queue

(hot, 0), (dot, 1), (lot, 1) ..

} } building: O(WL)

• b neighbours per node on average

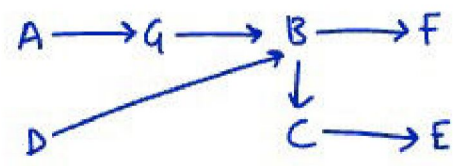
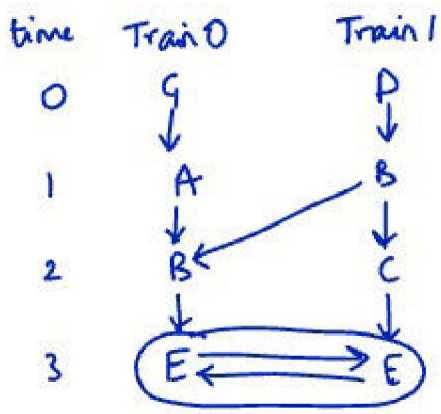


• end word dist(d)
 b^d nodes
 $O(1 + b + b^2 + \dots + b^{d-1})$
 $\sim O(b^d)$

Bidirectional
 BFS



$b^{d/2} + b^{d/2}$
 $\sim O(b^{d/2})$



Each vertex in the graph corresponds to a unique pair station {time}

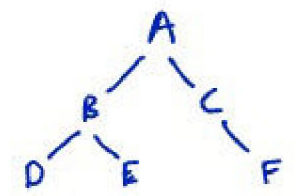
B: {2, 1, 4}

E: {3}

→ [1, 2, 4]

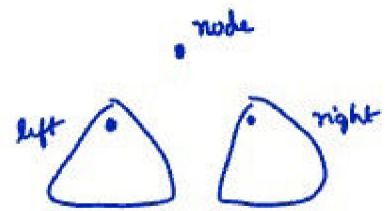
(B, 1) → (B, 2)

(B, 2) → (B, 4)

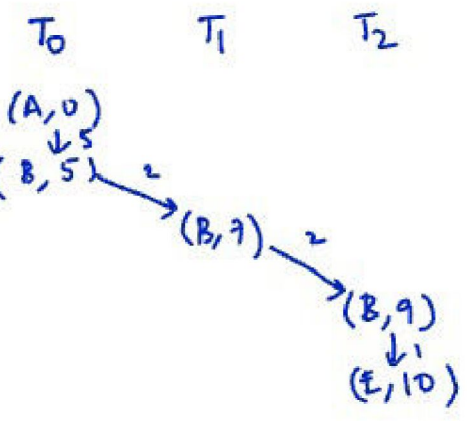


preorder: A B D E C F

Inorder: D B E A C F



- level 0
- level 1
- level k



Horner's method $ax^2 + bx + c$

$((a)x + b)x + c$

350

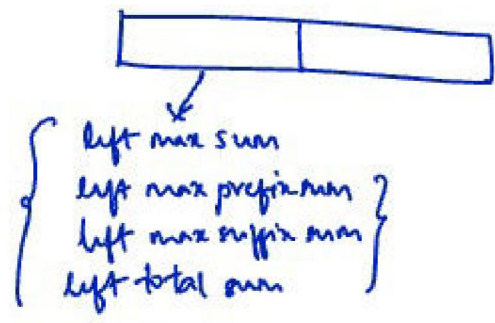
<inorder num> # <inorder string> <preorder num> # <preorder>

l_1 = left-max-path sum ending at root ✓ r_1

l_2 = left-max-path sum not ending at root r_2

$r_1 = \max\{\text{node.val}, l_1 + \text{node.val}, r_1 + \text{node.val}\}$

$r_2 = \max\{l_2, r_2, l_1, r_1, l_1 + \text{node.val} + r_1\}$

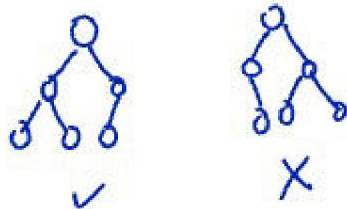


Trees:

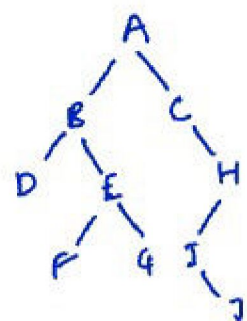
- Binary trees: 0/1/2 children

- Complete binary trees:

- Every level except the last level is filled
- last level - either filled from left to right
- Heaps



- We can maintain complete binary trees as arrays.
- height is $\Theta(\log n)$



Preorder: A B D E F G C H I J

Postorder: D F G E B I H C A

Inorder: D B F E G A C I J H

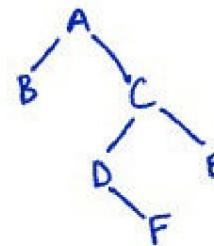
Level order: A B C D E H F G I J

On BSTs is the sorted order of all elements

Inorder + Preorder/post order can uniquely determine the binary tree

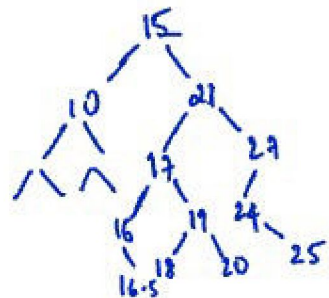
Postorder: [B] [F D] [E] C A

Inorder: [B] A [D F] C [E]



BSTs:

- For every node, all elements in the left subtree are smaller than the node & all elements in the right subtree are greater than the node



Heaps (Min heaps)

- Every node is $\geq$ its parent

height is $\Theta(\log n)$

Balanced BSTs:

- AVL trees $\rightarrow$ add ht to each node
- Red Black trees $\rightarrow$ add color to each node
- 2-3 trees

Delete:

- 3 cases

1. No children

2. One child

3. Two children

1. Delete successor from right subtree (or predecessor from left subtree) — guaranteed to be case 1 or case 2

2. Replace node to delete with deleted node from step 1

Symbol table:

- ADT

operations:

insert(key, value)

delete(key)

exists(key)

- Data Structures: Hash table, BSTs, Sorted arrays etc

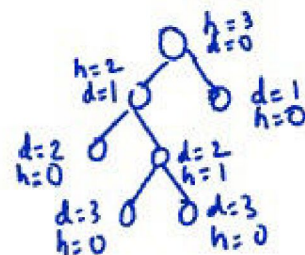
Operation:

- rank: (inverse of select) find its position in sorted order
- select: finding kth smallest / kth largest

Augmenting

- add more info in each node
- add count (# elements in subtree rooted at that node) to each node

def select(node, k):



Abstract data types (ADTs)

- API

- What ops can be done on it

Eg: Priority Queue

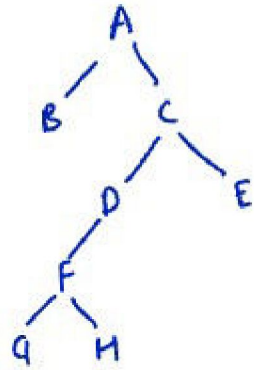
Data Structure:

- Actual implementation

Eg: Sorted array,
Unsorted array,
linked list,
Min/Max Heap

- Insert elements randomly: Expected height is $\Theta(\log n)$

- Equivalent to Shuffling array before quicksort



Queue
H A ~~F~~ ~~E~~ ~~D~~ ~~G~~ ~~H~~

A B C D E F G H

- No need to maintain a visited set as there are no cycles in trees (unlike general graphs)

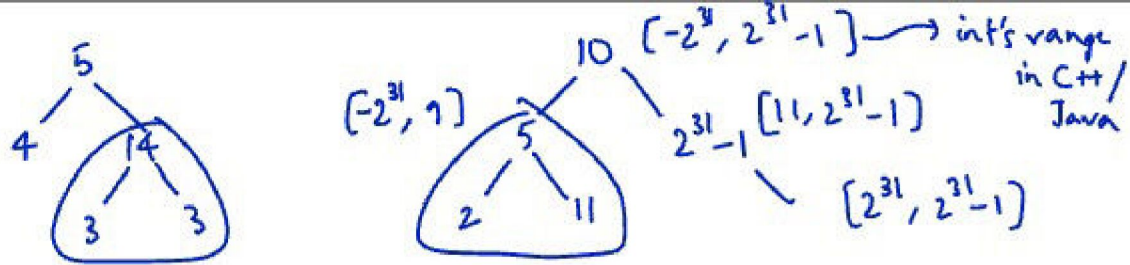
- Layer by layer

- Invariant: one layer (level) of nodes is in the queue.

 A

 F E

 G D



Horner's Method:

- Most efficient way to evaluate polynomials

$$an^3 + bn^2 + cn + d$$

$$\underbrace{\underbrace{\underbrace{((0 \cdot n + a)n + b)n + c}_{{an^2 + bn + c}}n + d}_{an^3 + bn^2 + cn + d}}$$

1435 -

0

$$0 \cdot 10 + 1 = 1$$

$$1 \cdot 10 + 4 = 14$$

$$14 \cdot 10 + 3 = 143$$

$$143 \cdot 10 + 5 = 1435$$

Tree: A graph G is a tree ^(n vertices)

- G is connected & acyclic (definition)
- G is connected & has exactly $n-1$ edges
- G is acyclic & has exactly $n-1$ edges
- Every pair of vertices has a unique path
- G is minimally connected (removing any edge ^{every edge is a cut edge} makes it disconnected)
- G is maximally acyclic (adding any edge creates a cycle)